ANALIZADOR SINTÁCTICO PASCAL – C

DOCUMENTACIÓN DEL ASIGNACIÓN ENTRE LOS INTEGRANTES

ALGORITMO DE TABLA DE ANÁLISIS SINTÁCTICO

Equipo 6

Integrantes:

Sánchez Aguilar Luis Ángel
Álvarez Olivera Irving Damián
Ramírez Manzano Jaret
García Barela Ángel Andrés
López Osorio Haniel
Martinez Cesar Zaid
Soriano Martínez Miguel Ángel

Materia: Compiladores

Profesor: David Martínez Torres

Fecha de entrega: Viernes 23 de octubre del 2026

UNIVERSIDAD TECNOLÓGICA DE LA MIXTECA

Índice

1. Introducción
2. Descripción general del sistema
3. Objetivo del programa
4. Arquitectura del sistema
    - 4.1. Entrada y procesamiento de la colección canónica
    - 4.2. Construcción de la tabla de análisis sintáctico
    - 4.3. Acciones de desplazamiento
    - 4.4. Acciones de reducción
    - 4.5. Acción de aceptación y manejo de errores
    - 4.6. Interfaz gráfica
    - 4.7. Generación de resultados
5. Tecnologías utilizadas
6. Flujo de funcionamiento
7. Conclusiones
8. Asignaciones de la semana
9. Evidencias

1. Introducción

El presente documento describe el desarrollo de un programa destinado a la construcción de una tabla de análisis sintáctico, una de las etapas fundamentales en la implementación de analizadores sintácticos dentro del área de compiladores.

El programa utiliza la información de una gramática y de su colección canónica de elementos LR(0) para determinar las acciones que deben realizarse durante el análisis de una secuencia de entrada. Estas acciones permiten establecer cuándo se deben desplazar símbolos, reducir mediante una producción, aceptar una cadena o reportar un error sintáctico.

Asimismo, el sistema organiza las acciones y transiciones en una tabla que facilita la consulta de los estados y símbolos correspondientes. De esta manera, se busca automatizar el procedimiento de construcción de la tabla y facilitar la comprensión del funcionamiento del análisis sintáctico.

2. Descripción general del sistema

El sistema funciona como una herramienta de apoyo para la construcción de la tabla de análisis sintáctico a partir de la información obtenida durante la construcción de la colección canónica.

Su funcionamiento se basa en recorrer los estados generados previamente y analizar los elementos que contiene cada uno. A partir de estos elementos y de las transiciones entre estados, se determinan las acciones que deben registrarse en la tabla para los símbolos terminales y no terminales de la gramática.

La tabla de análisis sintáctico se organiza mediante dos componentes principales: **ACTION** y **GOTO**. La primera permite establecer las acciones que se realizarán cuando se encuentre un símbolo terminal, mientras que la segunda indica el estado al que se debe avanzar cuando se completa una reducción y se consulta el símbolo no terminal correspondiente.

De manera general, el procesamiento sigue el siguiente esquema:

Gramática → Colección canónica → Identificación de acciones y transiciones → Construcción de la tabla → Visualización de resultados

El resultado es una tabla que concentra la información necesaria para dirigir el proceso de análisis sintáctico de acuerdo con los estados y símbolos definidos por la gramática.

3. Objetivo del programa

El objetivo principal del programa es automatizar la construcción de una tabla de análisis sintáctico a partir de la colección canónica de elementos LR(0), organizando las acciones y transiciones necesarias para representar el comportamiento del analizador.

Entre sus principales funciones se encuentran:

- Procesar la información de la gramática y de la colección canónica.
- Identificar los estados y símbolos que intervienen en el análisis.
- Registrar las acciones de desplazamiento y reducción.
- Identificar la acción de aceptación cuando corresponda.
- Establecer las transiciones entre estados mediante la sección GOTO.
- Organizar las acciones en una tabla de análisis sintáctico.
- Detectar posibles conflictos entre las acciones calculadas.
- Mostrar los resultados mediante una representación organizada.
- Verificar la tabla generada mediante diferentes gramáticas de prueba.

4. Arquitectura del sistema

La arquitectura del programa se organiza en módulos que permiten procesar la información de la colección canónica y convertirla en una tabla de análisis sintáctico.

Cada componente realiza una tarea específica dentro del procedimiento, desde la lectura de los estados y sus transiciones hasta la identificación de las acciones correspondientes y la presentación de los resultados.

Esta organización permite separar los cálculos de la tabla de su representación visual, facilitando la revisión y corrección de los resultados.

4.1. Entrada y procesamiento de la colección canónica

Esta etapa se encarga de obtener la información necesaria para construir la tabla de análisis sintáctico. Se utilizan los estados de la colección canónica, los elementos LR(0) que contiene cada estado y las transiciones registradas entre ellos.

La información permite identificar los símbolos que pueden procesarse desde cada estado y determinar las acciones que deben incorporarse a la tabla.

Si el programa utiliza directamente los módulos del proyecto anterior, esta etapa puede reutilizar la estructura de la gramática y la colección canónica previamente generada.

Entrada: colección canónica, elementos LR(0), transiciones y símbolos de la gramática.

Salida: información organizada para determinar las acciones y transiciones de la tabla.

4.2. Construcción de la tabla de análisis sintáctico

La construcción de la tabla consiste en relacionar cada estado de la colección canónica con los símbolos terminales y no terminales de la gramática.

Para ello, se organiza una estructura en la que cada fila representa un estado y las columnas corresponden a los símbolos que pueden intervenir en el análisis. Las columnas de los terminales se utilizan para registrar las acciones de ACTION, mientras que las de los no terminales se destinan a las transiciones de GOTO.

La información de cada celda se obtiene mediante el análisis de los elementos y las transiciones correspondientes al estado.

Este procedimiento permite concentrar en una sola estructura las decisiones que debe tomar el analizador sintáctico durante el procesamiento de una cadena.

Entrada: estados de la colección canónica y sus transiciones.

Salida: tabla organizada por estados, terminales y no terminales.

4.3. Acciones de desplazamiento

Las acciones de desplazamiento, comúnmente representadas mediante s o shift, indican que el símbolo terminal de entrada puede reconocerse como parte de una transición hacia otro estado.

Cuando un estado tiene una transición mediante un símbolo terminal, dicha transición se registra en la sección ACTION como una acción de desplazamiento hacia el estado correspondiente.

Por ejemplo, si desde el estado (I_0) existe una transición con el terminal id hacia el estado (I_3), la celda correspondiente a ACTION[(I_0), id] puede registrar s3.

Estas acciones permiten que el analizador avance en la entrada y continúe el procesamiento de la cadena.

Entrada: estado actual y transición mediante un símbolo terminal.

Salida: acción de desplazamiento hacia el estado sucesor.

4.4. Acciones de reducción

Las acciones de reducción permiten reconocer que una secuencia de símbolos corresponde al lado derecho de una producción gramatical y que puede sustituirse por el no terminal de su lado izquierdo.

Para identificarlas, se revisan los elementos LR(0) de cada estado. Cuando un elemento presenta el punto al final de una producción, indica que dicha producción se encuentra completa.

En una tabla LR(0), esta condición permite registrar una acción de reducción en las columnas de ACTION correspondientes a los terminales, de acuerdo con las reglas del algoritmo utilizado.

Por ejemplo, si el elemento completo corresponde a (A \rightarrow \alpha \cdot), la tabla puede registrar una reducción mediante la producción (A \rightarrow \alpha).

Estas acciones permiten al analizador reconocer estructuras gramaticales y continuar el proceso a partir del no terminal resultante.

Entrada: elementos LR(0) que contienen producciones completas.

Salida: acciones de reducción asociadas a las producciones correspondientes.

4.5. Acción de aceptación y manejo de errores

La acción de aceptación indica que la cadena de entrada cumple con las condiciones establecidas para finalizar satisfactoriamente el análisis sintáctico.

En el procedimiento LR(0), esta acción se identifica a partir del elemento completo de la producción inicial aumentada, siempre que se cumplan las condiciones establecidas por el algoritmo de análisis.

Por otra parte, las celdas para las cuales no se determina una acción válida representan situaciones en las que el analizador no puede continuar con la operación esperada. Estas posiciones pueden mantenerse vacías o identificarse como errores, según la representación utilizada por el programa.

También es importante revisar si dos o más reglas producen acciones diferentes para una misma celda, ya que esto puede generar conflictos que deben identificarse durante la construcción de la tabla.

Entrada: elementos completos, estados y acciones calculadas.

Salida: acción de aceptación, celdas sin acción válida e identificación de posibles conflictos.

4.6. Interfaz gráfica

La interfaz gráfica permite visualizar la tabla de análisis sintáctico de manera organizada y facilita la consulta de las acciones registradas para cada estado y símbolo.

Mediante esta interfaz, el usuario puede acceder a la información generada por el programa y revisar la correspondencia entre los estados de la colección canónica y las acciones calculadas.

La presentación de la tabla debe permitir distinguir las columnas correspondientes a ACTION y GOTO, así como identificar las acciones de desplazamiento, reducción y aceptación.

De esta manera, la interfaz funciona como un medio para consultar los resultados y comprobar visualmente la estructura de la tabla.

4.7. Generación de resultados

Una vez procesada la colección canónica, el programa presenta la tabla de análisis sintáctico con las acciones y transiciones determinadas para cada estado.

Los resultados permiten revisar qué operación corresponde a cada combinación de estado y símbolo. Asimismo, la tabla facilita la identificación de celdas vacías, acciones de aceptación y posibles conflictos derivados de la gramática.

La información generada puede utilizarse para comprobar el procedimiento de construcción de la tabla y como base para una etapa posterior de análisis de cadenas, si esta se incorpora al sistema.

Resultado: tabla de análisis sintáctico organizada, con acciones y transiciones calculadas a partir de la colección canónica.

5. Tecnologías utilizadas

Para el desarrollo del sistema se consideran las herramientas y componentes empleados en la implementación de los algoritmos y en la presentación de sus resultados.

Tecnología o componente	Función
Python	Lenguaje utilizado para implementar los algoritmos y controlar el procesamiento.
Estructuras de datos	Representar los estados, las producciones, los símbolos y las acciones de la tabla.
Colección canónica LR(0)	Proporcionar los estados y las transiciones necesarios para construir la tabla.
Tabla ACTION	Registrar las acciones de desplazamiento, reducción, aceptación y las posiciones sin acción válida.
Tabla GOTO	Registrar las transiciones asociadas a los símbolos no terminales.
Interfaz gráfica	Facilitar la visualización y consulta de la tabla generada.
Gramáticas de prueba	Comprobar los resultados obtenidos mediante diferentes casos.

6. Flujo de funcionamiento

El funcionamiento general del programa puede describirse mediante las siguientes etapas:

1. Se obtiene la gramática y la colección canónica generada previamente.
2. Se identifican los estados, elementos LR(0), terminales, no terminales y transiciones.
3. Se prepara la estructura de la tabla, organizando las filas por estados y las columnas por símbolos.
4. Se registran las acciones de desplazamiento a partir de las transiciones mediante terminales.
5. Se identifican los elementos con producciones completas para determinar las acciones de reducción.
6. Se identifica la condición de aceptación de acuerdo con la producción inicial aumentada.
7. Se registran las transiciones correspondientes a los no terminales en la sección GOTO.
8. Se revisan las celdas sin acción válida y los posibles conflictos.
9. Se presenta la tabla de análisis sintáctico mediante la interfaz gráfica.
10. Se comprueban los resultados con las gramáticas y los casos de prueba establecidos.

De esta manera, el programa transforma la información de la colección canónica en una tabla que organiza las decisiones necesarias para dirigir un análisis sintáctico basado en estados.

7. Conclusiones

El desarrollo del programa de generación de la tabla de análisis sintáctico permite continuar con el procedimiento de construcción de analizadores sintácticos iniciado mediante la colección canónica de elementos LR(0).

A través del procesamiento de los estados y las transiciones, es posible identificar las acciones de desplazamiento, reducción y aceptación, así como organizar las transiciones entre estados mediante la sección GOTO.

La separación de estas operaciones en módulos facilita la comprensión del procedimiento y permite verificar cada parte antes de integrar los resultados en la tabla final.

Finalmente, la revisión de las acciones generadas y la comparación con los resultados esperados constituyen aspectos importantes para comprobar la consistencia de la tabla. El proyecto permite aplicar los conceptos estudiados en la materia de Compiladores y comprender cómo la colección canónica sirve de base para determinar las acciones de un analizador sintáctico.

8. Asignaciones de la semana

DÍA	TAREA ASIGNADA	DESCRIPCIÓN	RESPONSABLE
Lunes 19	Organización y definición de gramáticas	Dividir las asignaciones del proyecto según corresponde y analizar sus requerimientos. Desarrollar las diferentes gramáticas para el desarrollo posterior de los algoritmos.	Sanchez Aguilar Luis Angel Alvare Olivera Irving Damian
Martes 20	Desarrollo de algoritmos y módulos de las respectivas gramáticas	Implementar el algoritmo que permite construir la tabla de análisis sintáctico LR, a partir de la colección canónica, la gramática aumentada y el algoritmo de PRIMEROS y SIGUIENTES.	Garcia Barela Angel Andres Sanchez Aguilar Luis Angel
Miércoles 21	Implementación de la interfaz	Comenzar con el diseño técnico de la interfaz gráfica donde se ejecutarán los algoritmos.	Ramirez Manzano Jaret Soriano Martinez Miguel Angel
Jueves 22	Pruebas y ERRORES	Registrar posibles fallas y trabajar en su debida corrección. Ademas de posibles fallas de compatibilidad.	Lopez Osorio Haniel Martinez Cesar Zaid
Viernes 23	Entrega Final	Terminar de registrar las asignaciones de cada integrante y organizar el respectivo documento al respecto. Reunir todas las aportaciones del equipo.	Martinez Cesar Zaid Soriano Martinez Miguel Angel

9. Evidencias

A continuación, se muestran pruebas del proceso para realizar la tabla de análisis sintáctico.

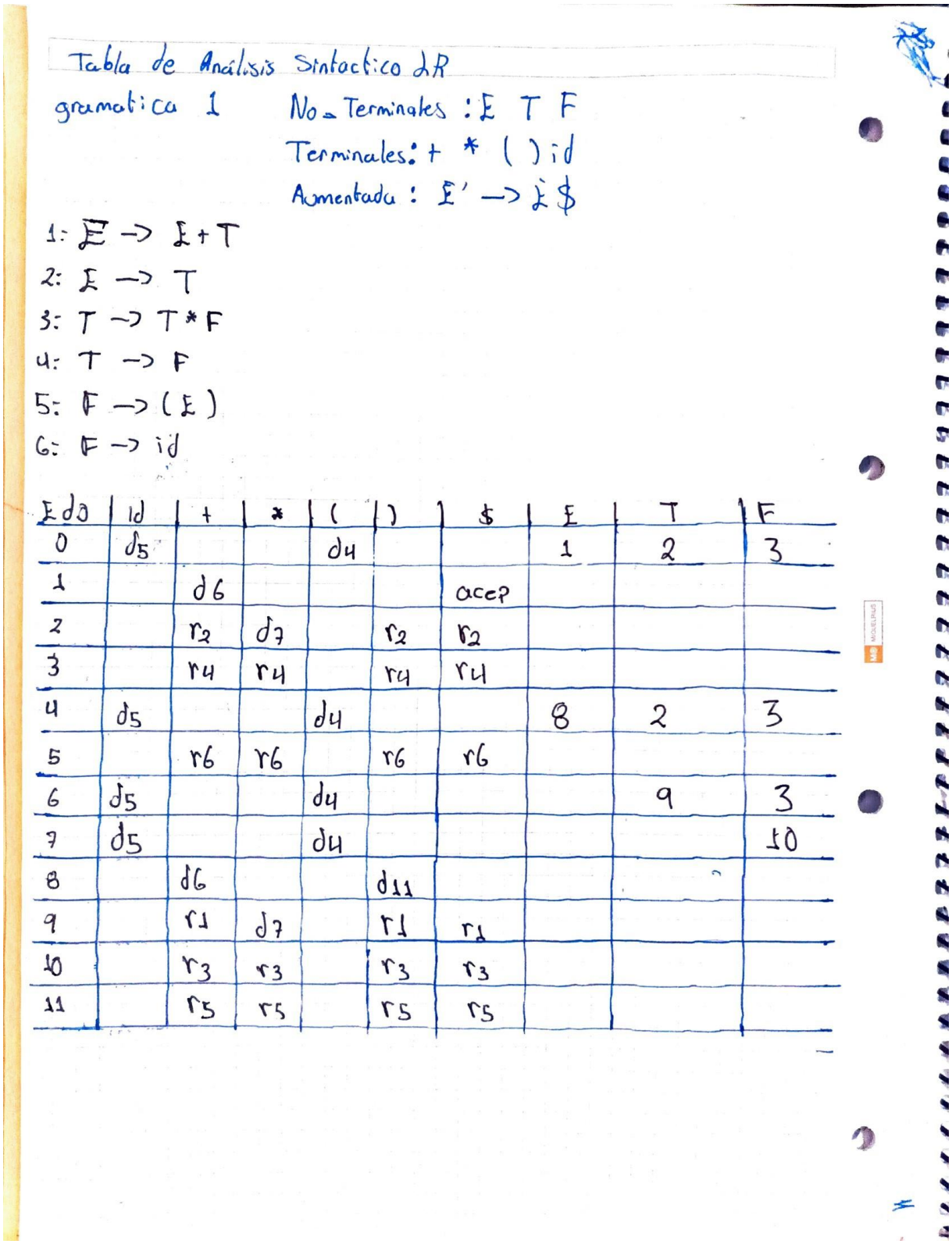


Tabla de Análisis Sintactico LR

gramatica 1

No_Terminales : E T F

Terminales: + * ( ) id

Aumentada : E' -> E $

1: E -> E+T

2: E -> T

3: T -> T*F

4: T -> F

5: F -> (E)

6: F -> id

Edo	id	+	*	(	)	$	E	T	F
0	d5			d4			1	2	3
1		d6				acep			
2		r2	d7		r2	r2			
3		r4	r4		r4	r4			
4	d5			d4			8	2	3
5		r6	r6		r6	r6			
6	d5			d4				9	3
7	d5			d4					10
8		d6			d11				
9		r1	d7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

Tabla de Analisis Sintactico LR

Gramatica N. 2

D' -> D$
1: D -> DD
2: D -> T id L ;
3: L -> , id L
4; L -> ϵ
5: T -> float
6: T -> int

No_Terminales: D L T

Terminales: $ id ; , ϵ float int

Edos	id	;	,	float	int	$	D	L	T
0				d3	d4	accpt	1		2
1				d3	d4		5		2
2	d6								
3	r5								
4	r6								
5				d3 r1	d4 r1	r1	5		2
6		r4	d8						7
7		d9							
8	d10								
9				r2	r2	r2			
10			d8					11	
11		r3							

Gramatica No 3

D' -> D $

1: D -> T L ;

2: L -> id L

3: L -> , id L

4: L -> ϵ

5: T -> float

6: T -> int

7: T -> ϵ

No_Terminales: D L T

Terminales : ; , id float int

Edos	id	;	,	float	int	$	D	L	T
0	r7	r7	r7	d3	d4		1		2
1						accpt.			
2	d7	r4	d5					6	
3	r5	r5	r5						
4	r6	r6	r6						
5	d8								
6		d9							
7	d7	r4	d5					10	
8	d7	r4	d5					11	
9						r1			
10		r2							
11		r3							

Gramatica 4

No_Terminal: S E 2

Terminal: id print ( ) num ; := + ,

S' -> S $

1: S -> S ; S
2: S -> id := E
3: S -> print (2)
4: E -> id
5: E -> num
6: E -> E + E
7: E -> (S , E)
8: 2 -> E
9: 2 -> 2 , E

Edos	id	print	(	)	num	;	:=	+	,	S	E	2	$
0	d2	d3							d3	1			
1						d4							acept
2							d5						
3			d6										
4	d2	d3								7			
5	d10		d8		d11						9		
6	d15		d12		d16						13	14	
7						d4 r7	d			17			r1
8	d18	d19								17			
9						r2		d20					r2
10						r4		r4					r4
11	d18												
12													
13													
14													
15													
16													
17													
18													
19													
20													
21													
22	d15		d12								21		

Gramatica No—5

S'—> S $

1: S —> T V ;

2: V —> id V'

3: V' —> [nint] V'

4: V —> ∈

5: T —> Int

6: T —> char

7: T —> float

No_Terminales: S V V' T

Terminales : ; id [ ] nint Int char float

Edos	id	;	[	]	Int	char	float	$	T	S	V	V'
0					d5	d3	d4		2	1		
1								acept.				
2	d7										6	
3	r6											
4	r7											
5	r5											
6		d8										
7		r4	d10									9
8								r1				
9		r2										
10					d11							
11				d12								
12		r4	d10									13
13		r3										